



Listas em Python: Fatiamiento, Dicionários, Dataclass e Cópia Rasa vs Profunda

O que são listas?

Uma lista em Python é uma **coleção ordenada de objetos**. Os elementos podem se repetir e são acessados por índice, começando em zero. Cada elemento pode ter um tipo diferente, tornando as listas extremamente versáteis.

```
my_list = [1, "a", 3.5, True]
my_list
```

Saída esperada:

```
[1, 'a', 3.5, True]
```

Criando listas

Sintaxe básica

Listas são criadas com colchetes [], separando os elementos por vírgulas

Tipos misturados

Podem conter arrays NumPy, strings e números ao mesmo tempo

Aninhamento

É possível aninhar listas dentro de listas para estruturas complexas



```
import numpy as np
weight = np.array([60, 72, 57, 90, 95, 72])
height = np.array([1.75, 1.80, 1.65, 1.90, 1.74, 1.91])
subject = ["A", "B", "C", "D", "E", "F"]
mybag = [weight, height, subject, 0, "a"]
len(mybag), type(mybag)
```

Saída esperada: (5, <class 'list'>)

Conteúdo da lista

Uma lista pode conter **múltiplos objetos de tipos diferentes**. Cada posição mantém o tipo original do objeto armazenado.

Os elementos são acessados pelo índice, permitindo navegação precisa pela estrutura de dados.



```
for i, item in enumerate(mybag):  
    print(i, type(item))
```

Saída esperada:

```
0 <class 'numpy.ndarray'>  
1 <class 'numpy.ndarray'>  
2 <class 'list'>  
3 <class 'int'>  
4 <class 'str'>
```

Fatiamento de listas

Indexação zero-based

O primeiro elemento está na posição 0

Padrão de fatiamento

Início incluso e fim exclusivo

Exemplo prático

`[0:3]` retorna os elementos nas posições 0, 1 e 2



```
mybag[0:3]
```

Retorna: array de peso, altura e lista de sujeitos



```
mybag[:2]
```

Retorna: apenas arrays de peso e altura

Elemento vs sublista

Acesso por índice único

Com um **índice único**, obtemos o **objeto diretamente**

```
mybag[0] # Retorna: numpy.ndarray
```

Acesso por intervalo

Com um **intervalo**, obtemos uma **sublista** com os elementos selecionados

```
mybag[0:1] # Retorna: list
```

Exemplo completo:

```
print(type(mybag[0:1]), mybag[0:1])
print(type(mybag[0]), mybag[0])
```

Saída:

```
<class 'list'> [array([60, 72, 57, 90, 95, 72])]
<class 'numpy.ndarray'> [60 72 57 90 95 72]
```

Essa diferença é **fundamental** para entender a estrutura dos dados e evitar erros comuns.

Dicionários: coleções com nomes

Um dicionário (dict) associa chaves a valores. É ideal quando queremos acessar elementos pelo nome em vez da posição. Os valores podem ser de tipos diferentes.



```
bag = {  
    "weight": weight,  
    "height": height,  
    "subject": subject,  
}  
bag["weight"]    # array([60, 72, 57, 90, 95, 72])  
bag["nome"] = "a" # adiciona nova chave  
list(bag.keys()) # ['weight', 'height', 'subject', 'nome']
```

Use lista quando a posição importa; use dict quando o nome importa.

Registros com dataclass

Quando um conjunto de dados tem **campos fixos e semânticos**, Python usa `dataclass` — um tipo com atributos nomeados, acessados por ponto.



```
from dataclasses import dataclass
import numpy as np
```

```
@dataclass
```

```
class Bag:
```

```
    weight: np.ndarray # peso
```

```
    height: np.ndarray # altura
```

```
    subject: list      # sujeitos
```

```
    valor: int         # valor
```

```
    nome: str          # nome
```

```
mybag = Bag(weight, height, subject, 0, "a")
```

```
mybag.weight
```

Saída: `array([60, 72, 57, 90, 95, 72])`

Campos opcionais

Campos com valor padrão

Declare na classe todos os campos que o objeto pode ter, inclusive os opcionais, com um valor padrão.

Marcar como ausente

Use `None` (com o tipo `X | None`) para indicar um campo ainda não preenchido.

Manter consistência

Atributos criados fora da classe não aparecem na representação nem na comparação de objetos.



```
from dataclasses import dataclass
import numpy as np
@dataclass
class Bag:
    weight: np.ndarray
    height: np.ndarray
    subject: list
    valor: int | None = None
    nome: str | None = None
    bmi: np.ndarray | None = None

mybag = Bag(weight, height, subject)
mybag.bmi = mybag.weight / mybag.height**2
mybag.bmi.round(2)
# array([19.59, 22.22, 20.94, 24.93, 31.38, 19.74])
```

Indexação e mutabilidade

Mutabilidade por referência

Listas e objetos são **mutáveis por referência**. Múltiplas variáveis podem apontar para o mesmo objeto na memória.

Cópia rasa

A cópia com `.copy()` é **rasa**: objetos internos continuam compartilhados.

Alterações em objetos internos **afetam todas as referências**.



```
mybag_list = [weight, height, subject]
a = mybag_list
b = mybag_list.copy()
a[0][0] = 999
a[0][0], b[0][0]    # (999, 999)
```



```
import copy
c = copy.deepcopy(mybag_list)
a[0][0] = 60
c[0][0], weight[0]  # (999, 60)
```

a e b compartilham o mesmo array, que é o próprio weight. Para uma cópia totalmente independente, use `copy.deepcopy()`.

Referências

Think Python

Downey, A. *Think Python: How to Think Like a Computer Scientist*. O'Reilly Media.

Uma introdução essencial aos fundamentos de programação e pensamento computacional usando Python.

Python Data Science Handbook

VanderPlas, J. *Python Data Science Handbook*. O'Reilly Media.

Um guia abrangente para as ferramentas essenciais de trabalho com dados em Python, incluindo NumPy, Pandas e bibliotecas de visualização.

Data Science from Scratch

Grus, J. *Data Science from Scratch*. O'Reilly Media.

Aprenda os fundamentos de ciência de dados construindo algoritmos e ferramentas do zero usando Python.